Introduction to Software Engineering

Software Testing

The student team is required to complete the **Software Testing** documentation for the assigned course project, following the attached template.

Software Engineering Department
Faculty of Information and Technology
University of Science

Table of Contents

Software Testing

Objectives

This document focus on the following topics:

- ✔ Completing the Software Testing document with the following sections:
  - Test Plan
  - Test Cases
- ✔ Understanding the Software Testing document.

1 Member Contribution Assessment

ID	Name	Contribution (%)	Signature
<Student1>			
<Student2>			
<Student3>			
<Student4>			

2 Test plan

2.1 Testing Scope & Objectives

The primary objective of this testing plan is to establish a structured Verification & Validation (V&V) process for the **Omni Platforms** web application. The scope covers both functional features (Web UI, RESTful APIs) and project documentation (SRS, Database Schema, UI Mockups):

- **Functional Scope:** Verifying core business features including Authentication & RBAC, Workspace & Team Management, AI Post Management & Workflow, Task & Calendar scheduling, and Distribution Channel configuration.
- **Quality Objectives:** Ensuring all implemented Use Cases strictly satisfy functional requirements, validation constraints, and role permissions defined in the project specification.

2.2 Requirement Traceability Matrix

The matrix below directly maps all tested Use Cases to their corresponding functional modules, testing techniques, and verification tools :

Requirement / Use Case ID	System Module / Object	Test Objective	Applied Testing Technique	Verification Tool
U001	Authorization & Account	Verify OTP verification, user registration, and login authentication.	EP, BVA, Black-box API Testing	Postman, pytest
U002, U016, U017	Workspace Management	Verify workspace metadata retrieval, joining via PIN/UUID, and member roster view.	EP, BVA, Decision Table Testing	Postman, Web UI

U007, U009, U015	Post Management & Approval	Verify post creation, AI content generation, status editing, and Manager review flow.	State Transition, Requirements-Based Testing	Postman, React UI
U008, U018	Task & Team Calendar	Verify self-assigned task scheduling, deadline validation, and Manager task assignment.	Boundary Value Analysis (BVA), EP	React UI, pgAdmin
U012	Distribution Channels	Verify social channel disconnection and role-based access restrictions.	Negative Testing, RBAC Verification	React UI, Postman

2.3 Testing Applied to System Functions (Functional Objects)

2.3.1. Authorization & Role-Based Access Control (RBAC) Module (U001)

System Objects: /auth/verify_otp, /auth/register, and /auth/login API endpoints.

Techniques Applied:

- **Equivalence Partitioning (EP):** Partitioning valid and invalid input formats (e.g., standard email format vs. malformed email).
- **Boundary Value Analysis (BVA):** Testing boundary constraints on password length and OTP string formats.

2.3.2 Workspace & Team Management Module (U002, U016, U017)

System Objects: Workspace query API (GET /workspaces/{id}), Member join form, and Member list management UI.

Techniques Applied:

- **Boundary Value Analysis (BVA):** Verifying the exact 16-character length constraint for workspace_id and 4–8 digits for workspace_pin.
- **Decision Table Testing:** Verifying role-dependent privileges (e.g., Managers can view member rosters and controls, whereas Members only see read-only workspace details).

2.3.3. Post Creation, AI Integration & Approval Workflow (U007, U009, U015)

System Objects: Post editor canvas, AI prompt template selector, and Post status transition endpoint.

Techniques Applied:

- **Requirements-Based Testing:** Verifying that AI assistant toggles correctly populate prompt parameters and inject returned text into the editor canvas .
- **State Transition Testing:** Validating lifecycle state changes:

2.3.4. Task & Calendar Scheduling Module (U008, U018)

System Objects: Calendar interactive grid, Self-task modal, and Manager Task Assignment API

Techniques Applied:

- **Boundary Value Analysis (BVA):** Preventing task creation with empty titles or scheduling dates in the past relative to the current timestamp.
- **Negative Testing:** Verifying that the backend rejects assigning tasks to inactive or deleted workspace members.

2.3.5. Distribution Channel Configuration Module (U012)

System Objects: Connected social channels card UI, Disconnect confirmation modal, and Role access guard.

Techniques Applied:

- **Role-Based Access Control (RBAC) Testing:** Ensuring non-manager (Member) accounts cannot view or access channel configuration controls.

- **Business Rule Validation:** Blocking channel disconnection when active posts are currently linked or pending.

2.4 Document Objects (Static V&V)

Static V&V is conducted on documentation artifacts using Software Inspection and Review techniques :

- **Software Requirement Specification (SRS):** Inspected using checklists to verify requirement clarity, consistency, and testability across all Use Cases .
- **Database Schema & ERD:** Reviewed to ensure Foreign Key relationships, field type constraints, and cascading rules (ON DELETE CASCADE) are correctly defined.
- **UI/UX Figma Mockups:** Visually inspected against React component implementations to verify layout consistency and correct role-based views.
- **Source Code Base (/src/):** Evaluated via code reviews and linting to maintain coding standards and prevent basic logical errors .
    -
    -

2.5 Entry & Exit Criteria

2.5.1 Entry Criteria (Pre-requisites to begin testing)

- All core API endpoints and UI screens for target Use Cases are developed and deployed locally.
- Database schema is migrated with test seed data (sample users, workspaces, and posts).
- Test cases and expected outputs are clearly documented and reviewed .

2.5.2 Exit Criteria (Conditions to complete testing phase)

- 100% of defined test cases in Section 3 are executed .
- All critical functional bugs preventing core workflows (Login, Workspace join, Post submit) are identified, logged, and re-tested .
- Test execution summary accurately reflects Pass/Fail status with actual outputs recorded .

2.6 Risk Management & Contingency Plan

Risk ID	Risk Description	Likelihood	Impact	Mitigation Strategy	Contingency Plan
RSK-01	External AI Service API quota limit or network timeout during test execution.	Medium	High	Use mock responses (pytest-mock or mock JSON) to simulate AI generation text.	Fall back to manual static text entry in the editor canvas during offline testing.
RSK-02	Test data pollution or foreign key conflicts in local PostgreSQL database.	Medium	Medium	Maintain modular seed SQL scripts to reset test database tables before test runs.	Re-run automated migration and seed scripts to restore clean state.
RSK-03	Schedule slippage due to unexpected critical bugs found late in testing.	Low	High	Prioritize fixing blocker/critical defects in core flows (Auth, Post workflow) first.	Mark minor visual/UI cosmetic defects for future improvements.

3 Test cases

3.1 List of test cases

Seq	Test case	Target	Description
1	U001-TC01: Verify Email with OTP	Authentication API	Verify that the system accepts a valid OTP code and returns a temporary verification token.
2	U001-TC02: Register User Account	Registration API	Verify account and workspace creation upon submitting valid user credentials.
3	U001-TC03: User Login Authentication	Login API	Verify that correct credentials return a valid JWT access token and user profile.
4	U002-TC04: Retrieve Workspace Details	Workspace API	Verify that an authenticated user can retrieve workspace metadata.

5	U007-TC05: Create Draft Post	Post Creation API	Verify post creation and persistence in the database with initial "Draft" status.
6	U009-TC06: AI Content Generation	AI Editor Integration	Verify that AI generates post copy based on selected prompt template and context.
7	U015-TC07: Submit Post for Review	Post Workflow API	Verify state transition of a post from "Draft" to "Pending Review".
8	U016-TC08: Join Workspace with Valid PIN	Workspace Join Form	Verify that a member can join a workspace using valid Workspace ID and PIN.
9	U016-TC09: Join Workspace with Invalid PIN	Workspace Join Validation	Verify that invalid PIN or non-existent Workspace ID is rejected with an error message.
10	U017-TC10: View Workspace Member Roster	Team Workspace UI	Verify that the Manager can view workspace details and the active member list.

3.2 Test case specifications

3.2.1 Test case 1

Field	**Description**
Test case	U001-TC01: Verify Email with OTP

Related Use case	U001 (Register & Login)
Context	User enters their email address and OTP code to verify identity before registration.
Input Data	**Method / URL:** POST /auth/verify_otp **Body:** {"email": "user@example.com", "otp": "123456"}
Expected Output	**HTTP 200 OK** {"message": "Email verified successfully", "verification_token": "<token_string>", "expires_in": 900}
Test steps	1. Set HTTP request method to POST. 2. Set URL to BaseURL/auth/verify_otp. 3. Set request Body to raw JSON with valid email and OTP. 4. Send request and inspect response status and verification token.
Actual Output	HTTP 200 OK returned with valid verification token and 900s expiration.

Result	**Passed**

3.2.2 Test case 2

Field	Description
Test case	U001-TC02: Register User Account
Related Use case	U001 (Register & Login)
Context	User submits profile, password, verification token, and initial workspace details to create an account.
Input Data	**Method / URL:** POST /auth/register **Body:** {"username": "testuser", "email": "user@example.com", "password": "SecurePassword123!", "verification_token": "<token>", "account_type": "individual", "business_role": "manager", "workspace_name": "My Workspace", "workspace_id": "WS-TEST-UUID-01", "workspace_pin": "123456"}
Expected Output	**HTTP 201 Created** Returns user profile JSON object (users_uuid, username, email, created_at) and associated workspace object.

Test steps	1. Set method to POST. 2. Set URL to BaseURL/auth/register. 3. Populate payload with valid verification token and account information. 4. Send request and verify user persistence in database.
Actual Output	HTTP 201 Created returned; user and workspace records successfully generated.
Result	**Passed**

3.2.3 Test case 3

Field	**Description**
Test case	U001-TC03: User Login Authentication
Related Use case	U001 (Register & Login)
Context	Registered user submits email and password to log in and obtain an authentication session.

Input Data	**Method / URL:** POST /auth/login **Body:** {"email": "user@example.com", "password": "SecurePassword123!"}
Expected Output	**HTTP 200 OK** Returns JSON containing access_token (JWT Bearer string), token_type: "bearer", and authenticated user metadata.
Test steps	1. Set method to POST. 2. Set URL to BaseURL/auth/login. 3. Provide registered user credentials in JSON Body. 4. Send request and verify token issuance.
Actual Output	HTTP 200 OK returned with signed JWT access token.
Result	**Passed**

3.2.4 Test case 4

Field	**Description**

Test case	U002-TC04: Retrieve Workspace Details
Related Use case	U002 (Manage Workspace)
Context	Authenticated user requests workspace metadata using an authorized workspace ID.
Input Data	**Method / URL:** GET /workspaces/WS-TEST-UUID-01 **Headers:** Authorization: Bearer <valid_jwt_token>
Expected Output	**HTTP 200 OK** {"workspace_id": "WS-TEST-UUID-01", "workspace_name": "My Workspace", "manager_id": "<uuid>", "member_count": 1}
Test steps	1. Set method to GET. 2. Set URL to target workspace endpoint. 3. Include valid Bearer token in request header. 4. Send request and verify returned metadata.

Actual Output	HTTP 200 OK returned with accurate workspace details.
Result	**Passed**

3.2.5 Test case 5

Field	Description
Test case	U007-TC05: Create Draft Post
Related Use case	U007 / U009 (Create Post)
Context	Authenticated user creates a new social post and saves it in draft state.
Input Data	**Method / URL:** POST /posts **Headers:** Authorization: Bearer <valid_jwt_token> **Body:** {"workspace_id": "WS-TEST-UUID-01", "title": "Product Announcement", "content": "Official update...", "seo_keywords": ["launch", "tech"]}

Expected Output	**HTTP 201 Created** Returns post object with id, status: "draft", and timestamp metadata.
Test steps	1. Authenticate user and obtain token. 2. Send POST request to /posts with valid post payload. 3. Inspect response status code and post status attribute.
Actual Output	HTTP 201 Created returned; post successfully persisted as "draft".
Result	**Passed**

3.2.6 Test case 6

Field	**Description**
Test case	U009-TC06: AI Content Generation
Related Use case	U009 (Create & Edit Post with AI Integration)

Context	User requests AI assistant to generate post body text using selected prompt template.
Input Data	**AI Toggle:** ON **Prompt Template:** "Standard Social Post Summary" **Instruction:** "Highlight top 2 product features in bullet points"
Expected Output	AI service returns generated text; content is automatically populated inside the rich-text editor canvas.
Test steps	1. Open Content Editor module. 2. Toggle "Enable AI" switch to ON. 3. Select template and input extra instruction. 4. Click "Generate" and observe canvas rendering.
Actual Output	Editor canvas populated with relevant formatted bullet points within 2.5 seconds.
Result	**Passed**

3.2.7 Test case 7

Field	Description
Test case	U015-TC07: Submit Post for Review
Related Use case	U015 (Review & Approval Workflow)
Context	A member submits an existing draft post for Manager review.
Input Data	**Method / URL:** PATCH /workspaces/WS-TEST-UUID-01/posts/{post_id} **Headers:** Authorization: Bearer <member_token> **Body:** {"status": "pending_review"}
Expected Output	**HTTP 200 OK** Post status updates from draft to pending_review.
Test steps	1. Authenticate with Member account. 2. Send PATCH request with status = pending_review.

	3. Verify response status code and post status in database.
Actual Output	HTTP 200 OK returned; status updated to pending_review in database.
Result	**Passed**

3.2.8 Test case 8

Field	**Description**
Test case	U016-TC08: Join Workspace with Valid PIN
Related Use case	U016 (Join Team Workspace)
Context	Authenticated member joins a team workspace using credentials assigned by Manager.
Input Data	workspace_id: "WS-8849-A1B2-C3D4", workspace_pin: "123456"
Expected Output	Credentials validated; member associated with workspace in database and granted dashboard access.

Test steps	1. Log in with Member account. 2. Open "Join Workspace" modal. 3. Enter valid Workspace ID and PIN. 4. Click "Join" and verify redirection.
Actual Output	Member record linked to workspace; redirected to workspace dashboard.
Result	**Passed**

3.2.9 Test case 9

Field	**Description**
Test case	U016-TC09: Join Workspace with Invalid PIN
Related Use case	U016 (Join Team Workspace)
Context	Member enters an incorrect PIN or non-existent Workspace ID.

Input Data	workspace_id: "WS-8849-A1B2-C3D4", workspace_pin: "999999" (Incorrect PIN)
Expected Output	System rejects request and displays error: *"Invalid Workspace ID or Workspace Password"*.
Test steps	1. Open "Join Workspace" form. 2. Input valid Workspace ID and incorrect PIN. 3. Click "Join" and observe UI message.
Actual Output	Request rejected; error toast displayed with message *"Invalid Workspace ID or Workspace Password"*.
Result	**Passed**

3.2.10 Test case 10

Field	**Description**
Test case	U017-TC10: View Workspace Member Roster

Related Use case	U017 (Manage Team Workspace)
Context	Workspace Manager views workspace metadata and active member roster.
Input Data	User role: manager, workspace_id: "EWXDRDE7RY465RPC"
Expected Output	HTTP 200 OK returned; displays member list with name, role, email, and management action buttons.
Test steps	1. Log in as Workspace Manager. 2. Navigate to "Team Workspace" tab. 3. Inspect Workspace Details card and member list table.
Actual Output	Workspace details and active member roster rendered accurately with administrative controls.
Result	**Passed**

3.2.11 Test case 11

Field	Description
Test case	U018-TC11: Assign Task to Member
Related Use case	U018 (Assign Team Tasks)
Context	Manager creates and assigns a content publishing task to a team member.
Input Data	title: "Draft Product Launch Post", priority: "high", due_date: "2026-08-25T18:00:00Z", assigned_to: "<member_uuid>"
Expected Output	**HTTP 201 Created** Task record created in database with status todo and linked to assignee.
Test steps	1. Log in as Manager. 2. Click "Add Task" button in Team Workspace.

	3. Fill in Title, Priority, Due Date, and select Member. 4. Click "Submit" and verify task list.
Actual Output	Task created with HTTP 201; visible in team task table.
Result	**Passed**

3.2.12 Test case 12

Field	Description
Test case	U008-TC12: Create Self-Assigned Task
Related Use case	U008 (Personal Task & Calendar)
Context	User creates a personal task directly from the interactive Calendar grid.

Input Data	title: "Draft weekly newsletter", priority: "Medium", due_date: "2026-08-28 18:00"
Expected Output	Task is saved under personal context and rendered as a block indicator on the calendar cell.
Test steps	1. Open "Calendar" module from sidebar. 2. Click "+" button on a future date. 3. Enter Title, Priority, Deadline. 4. Click "Confirm" and check calendar grid.
Actual Output	Task saved in PostgreSQL and immediately rendered on target date cell.
Result	**Passed**

3.2.13 Test case 13

Field	**Description**
Test case	U008-TC13: Reject Past Deadline Task

Related Use case	U008 (Personal Task & Calendar)
Context	User attempts to create a task with a deadline set to a past date.
Input Data	title: "Late Task", due_date: "2026-08-10 10:00" (Past date)
Expected Output	Submission is blocked; UI highlights date picker in red with message: *"Cannot schedule tasks in past timelines."*
Test steps	1. Open "Add Task" modal. 2. Input valid title and select a past date. 3. Click "Confirm" and observe UI behavior.
Actual Output	Date input flagged; error message displayed and submission blocked.
Result	**Passed**

3.2.14 Test case 14

Field	**Description**

Test case	U012-TC14: Disconnect Channel Role Guard
Related Use case	U012 (Delete Distribution Channel)
Context	A Member account attempts to access channel distribution settings.
Input Data	User role: member, Target URL: /distribution/settings
Expected Output	Distribution tab is hidden in navigation; direct URL access is blocked and redirects to Dashboard with an unauthorized notice.
Test steps	1. Log in with Member account. 2. Verify absence of "Distribution" in sidebar. 3. Enter /distribution/settings directly in browser address bar. 4. Observe access control behavior.
Actual Output	Sidebar tab hidden; URL navigation redirected to /dashboard with 403 status message.
Result	**Passed**